

Capabilities(TASK 1)

1. Register/login (authenticate users)
2. Browse/search food items
3. Manage menu items (add/edit/delete) — admin
4. Add/remove items in cart
5. Place an order
6. Track order status
7. Update order status — admin
8. Submit a review for a food item

=====

Contracts (Task 3)

Order Service — Contract

Operation	Input	Output	Errors
placeOrder	customerId, cartId	orderId, totalPrice, status	InvalidCustomer, EmptyCart, ItemUnavailable
getOrderStatus	orderId	status	OrderNotFound
updateOrderStatus	orderId, newStatus	confirmation	OrderNotFound, InvalidStatus

Catalogue Service — Contract

Operation	Input	Output	Errors
browseItems	(none) or category	list of food items with prices	-
checkItemAvailability	itemId	true/false	ItemNotFound
manageMenuItem	itemId (for edit/delete), item details	confirmation	ItemNotFound, InvalidInput

User Service:

Operation	Input	Output	Errors
validateCustomer	customerId	true/false	CustomerNotFound

Cart Service:

Operation	Input	Output	Errors
getCartItems	cartId	list of items with quantities	CartNotFound

Review Service:

Operation	Input	Output	Errors
submitReview	customerId, itemId, rating, comment	reviewId	InvalidCustomer, ItemNotFound, InvalidRating

##placeOrder Full Specification (Task 4)

****Operation:**** placeOrder

****Input:****

- customerId — identifies which customer is placing the order
- cartId — identifies the customer's current cart

****Output (on success):****

- orderId — a unique identifier for the newly created order
- totalPrice — the calculated total cost of the order
- status — the initial status of the order (e.g. "Placed")

****Error cases:****

- InvalidCustomer — customerId does not correspond to a real, registered user
- EmptyCart — the cart has no items in it
- ItemUnavailable — one or more items in the cart are no longer available
- CartNotFound — the given cartId does not exist

****Internal steps this operation must hide from callers:****

1. Calling User Service (validateCustomer) to confirm the customer is real
2. Calling Cart Service (getCartItems) to fetch the items in the cart
3. Calling Catalogue Service (checkItemAvailability) for each item to confirm stock
4. Calculating the total price internally
5. Generating a new, unique orderId
6. Saving the new order and its items into the Order Service's own database tables
7. Clearing/marking the cart as converted (so it isn't reused for another order)

****Why these are hidden:**** the caller (e.g. a client app) only needs to send customerId + cartId and receive back an orderId + totalPrice + status.

It should never need to know HOW the order was validated, priced, or stored internally — that's the whole point of exposing this as one clean contract operation instead of many small steps.

##Validation Table (Task 6)

Service	Reachable	Self-Contained	Has a Contract	Independent	Loosely Coupled	Notes
User Service	Yes	Yes	Yes	Yes	Yes	Called by others but never calls out itself - cleanly independent.
Catalogue Service	Yes	Yes	Yes	Yes	Yes	A foundational service, no outgoing dependencies.
Cart Service	Yes	Yes	Yes	Yes	Yes	Calls Catalogue Service only through its contract, not its database.
Order Service	Yes	Yes	Yes	Yes	Yes	Calls User and Cart Service through contracts only - no

						shared tables.
Review Service	Yes	Yes	Yes	Yes	Yes	Calls User and Catalogue Service through contracts only - no shared tables.

Hypothetical failure case (for demonstration)

If Cart Service directly queried Catalogue Service's FoodItems table instead of calling checkItemAvailability, it would FAIL the "Loosely Coupled" and "Self-Contained" properties — because Cart Service would break the moment Catalogue Service changed its table structure, even if checkItemAvailability's contract never changed. The fix is exactly what we did: always go through the contract, never touch another service's internal data directly.
